

Strings

Strings in Python

A **string** in Python is an **ordered collection of characters** used to store and represent **text-based information**.

Strings can store:

- Words and sentences
- Names
- Text file contents
- URLs (Internet addresses)
- Python code
- Binary data (bytes)
- Unicode text (multiple languages)

Characteristics of Strings

1. Ordered Collection

- Characters are stored in a **left-to-right sequence**.
- Each character has a **position (index)**.

Example:

```
text = "Python"
```

Character	P	y	t	h	o	n
Index	0	1	2	3	4	5

2. Immutable

Strings are **immutable**, meaning:

- Once created, they **cannot be changed**.
- You cannot modify individual characters directly.
- If you change a string, Python creates a **new string** instead.

Example:

```
text = "Python"
```

```
# Not allowed
```

```
text[0] = "J"    # Error
```

Correct way:

```
text = "Jython"
```

3. Sequence Type

Strings belong to Python's **sequence** data types.

Because they are sequences, they support operations like:

- Indexing
- Slicing
- Iteration
- Length calculation
- Concatenation
- Membership testing

These operations also work on other sequence types like:

- Lists
- Tuples

Strings vs C Character Arrays

Python strings are **higher-level** than character arrays in languages like C.

Python

- Built-in string functions and methods.
- Easy to manipulate.
- No separate character data type.

C

- Strings are arrays of characters.
- Fewer built-in operations.
- Individual characters have a **separate char type**.

Characters in Python

Python **does not have a separate character type**.

A single character is simply a string of length 1.

Example:

```
ch = "A"
```

Type:

```
type(ch)
```

Output:

```
<class 'str'>
```

Empty String

An empty string contains **no characters**.

Common String Operations

Python supports many useful operations on strings.

1. Concatenation

Combining two or more strings.

```
first = "Hello"
```

```
second = "World"
```

```
result = first + " " + second
```

```
print(result)
```

Output:

Hello World

2. Indexing

Accessing a single character using its position.

```
text = "Python"
```

```
print(text[0])
```

Output:

P

3. Slicing

Extracting part of a string.

```
text = "Python"
```

```
print(text[1:4])
```

Output:

yth

String Methods

Python provides many built-in methods for common string tasks.

Examples:

- `upper()`
- `lower()`
- `strip()`
- `replace()`
- `split()`
- `find()`

Example:

```
text = "python"
```

```
print(text.upper())
```

Output:

PYTHON

Advanced String Processing

For more advanced text processing, Python provides modules such as:

- `re` (Regular Expressions) for pattern matching.
- Other text-processing libraries for specialized tasks.

Escape Sequences in Python Strings

Escape sequences are special character combinations that begin with a **backslash** (****). They are used to represent characters that are difficult or impossible to type directly from the keyboard.

An escape sequence consists of:

- A backslash (\)
- One or more characters following it

Python replaces the entire escape sequence with **one special character** in memory.

Why Use Escape Sequences?

Escape sequences allow you to:

- Insert new lines
- Add tabs
- Include quotes inside strings
- Insert backslashes
- Represent special ASCII or Unicode characters

Example

```
s = 'a\nb\tc'
```

Here:

- \n → Newline character
- \t → Horizontal tab

Length of a String

Use the len() function to count the actual number of characters stored in the string.

```
s = 'a\nb\tc'
```

```
print(len(s))
```

Output

5

The string contains **5 characters**:

Position	Character
1	a
2	Newline (\n)
3	b
4	Tab (\t)
5	c

Note: The backslash (\) is **not stored** in the string. It is only used while creating the string.

Common Escape Sequences

Escape Sequence	Meaning
\\	Backslash (\)
\'	Single quote (')
\"	Double quote (")
\n	Newline (Line break)
\t	Horizontal tab
\r	Carriage return

\b	Backspace
\f	Form feed
\v	Vertical tab
\a	Bell (alert sound)
\0	Null character
\xhh	Character with hexadecimal value hh
\ooo	Character with octal value ooo
\ followed by a newline	Line continuation (ignored)

Examples

1. Newline (\n)

```
print("Hello\nWorld")
```

Output

Hello
World

2. Tab (\t)

```
print("Python\tProgramming")
```

Output

Python Programming

3. Backslash (\\)

```
print("C:\\Users\\Admin")
```

Output

C:\Users\Admin

4. Single Quote (\')

```
print('It\'s Python')
```

Output

It's Python

5. Double Quote (\")

```
print("He said, \"Hello!\"")
```

Output

He said, "Hello!"

6. Hexadecimal Escape (\xhh)

```
print("\x41")
```

Output

A

(41 is the hexadecimal value for A.)

7. Octal Escape (\ooo)

```
print("\101")
```

Output

A

(101 is the octal value for A.)

Raw Strings in Python

A **raw string** is a string prefixed with **r** or **R** that tells Python **not to treat backslashes (\) as escape characters**.

In a raw string, backslashes are stored **exactly as they are typed**.

Syntax

```
r"string"
```

or

```
R"string"
```

Why Use Raw Strings?

Normally, Python interprets backslashes as **escape sequences**.

Example:

```
path = "C:\new\text.dat"
```

Python interprets:

- `\n` → Newline
- `\t` → Tab

So the string becomes:

C:

ew ext.dat

This is **not** the intended file path.

Using Raw Strings

Prefix the string with **r** to prevent escape sequence processing.

```
path = r"C:\new\text.dat"
```

```
print(path)
```

Output

C:\new\text.dat

Now Python keeps the backslashes exactly as written.

Example

```
normal = "Hello\nWorld"
```

```
raw = r"Hello\nWorld"
```

```
print(normal)
```

```
print(raw)
```

Output

Hello

World

Hello\nWorld

Triple-Quoted Strings (Multiline Strings) in Python

A **triple-quoted string** (also called a **block string**) is a string enclosed within **three single quotes ('')** or **three double quotes (""")**.

It allows you to write **multiline strings** easily without using newline escape sequences (\n).

Syntax

Using triple double quotes:

```
"""
```

Multiline

String

```
"""
```

Using triple single quotes:

```
'''
```

Multiline

String

```
'''
```

Both forms work the same way.

Why Use Triple-Quoted Strings?

Triple-quoted strings make it easy to:

- Write text across multiple lines.
- Preserve formatting exactly as typed.
- Avoid using multiple `\n` escape sequences.
- Include both single (') and double (") quotes without escaping them (in most cases).

Example

```
mantra = """Always look  
on the bright  
side of life."""
```

Quotes Inside Triple-Quoted Strings

You can include both **single quotes** and **double quotes** inside a triple-quoted string without escaping them.

Example:

```
text = """She said, "Python is fun!"  
It's easy to learn."""
```

Common Uses of Triple-Quoted Strings

1. Multiline Text

Useful for storing paragraphs or messages.

```
message = """Welcome!
```

```
This program demonstrates
```

```
Python strings."""
```

2. HTML or XML Code

You can embed HTML or XML directly.

```
html = """
```

```
<html>
```

```
<body>
```

```
<h1>Hello</h1>
```

```
</body>
```

```
</html>
```

```
"""
```

3. SQL Queries

```
query = """
```

```
SELECT *
```

```
FROM students
```

```
WHERE marks > 80;
```

```
"""
```

4. Documentation Strings (Docstrings)

Triple-quoted strings are commonly used as **docstrings** to document modules, functions, classes, and methods.

Example:

```
def greet():  
    """Returns a greeting message."""  
    return "Hello"
```

5. Temporarily Disabling Code (Commenting Out Code)

Developers sometimes use triple quotes to temporarily ignore a block of code during testing.

```
x = 1  
  
"""  
print("This code is temporarily disabled")  
print(x)  
"""  
  
y = 2
```

Note: This is a common practice during development, but triple-quoted strings are **not true comments**. Python creates a string object, whereas comments (#) are completely ignored.

String Functions

String Function	Syntax	Use	Example
len()	len(string)	Returns the length of a string	len("Python") → 6
lower()	string.lower()	Converts all characters to lowercase	"HELLO".lower() → "hello"
upper()	string.upper()	Converts all characters to uppercase	"hello".upper() → "HELLO"
title()	string.title()	Converts the first letter of each word to uppercase	"hello world".title() → "Hello World"
capitalize()	string.capitalize()	Capitalizes only the first character	"python".capitalize() → "Python"
strip()	string.strip()	Removes spaces from both ends	" hello ".strip() → "hello"
lstrip()	string.lstrip()	Removes spaces from the left	" hello".lstrip() → "hello"
rstrip()	string.rstrip()	Removes spaces from the right	"hello ".rstrip() → "hello"
replace()	string.replace(old, new)	Replaces one substring with another	"apple".replace("p","b") → "abble"
find()	string.find(sub)	Returns the index of the first occurrence; -1 if not found	"python".find("t") → 2

index()	string.index(sub)	Returns the index of the substring; gives an error if not found	"python".index("o") → 4
count()	string.count(sub)	Counts occurrences of a substring	"banana".count("a") → 3
startswith()	string.startswith(sub)	Checks if the string starts with the given text	"Python".startswith("Py") → True
endswith()	string.endswith(sub)	Checks if the string ends with the given text	"Python".endswith("on") → True
split()	string.split(separator)	Splits a string into a list	"a,b,c".split(",") → ['a','b','c']
join()	separator.join(list)	Joins list elements into a string	"-".join(['a','b','c']) → "a-b-c"
isalpha()	string.isalpha()	Checks if all characters are alphabets	"Hello".isalpha() → True
isdigit()	string.isdigit()	Checks if all characters are digits	"123".isdigit() → True
isalnum()	string.isalnum()	Checks if all characters are letters or digits	"ABC123".isalnum() → True
isspace()	string.isspace()	Checks if the string contains only spaces	" ".isspace() → True

swapcase()	string.swapcase()	Swaps uppercase to lowercase and vice versa	"PyThOn".swapcase() → "pYtHoN"
center()	string.center(width)	Centers the string within the given width	"Hi".center(6) → ' Hi '
zfill()	string.zfill(width)	Pads zeros on the left	"25".zfill(5) → "00025"

